

PyTorch and Automatic Differentiation: Worked Solutions

Problem 1. A reverse derivative product

Problem. For $f(x_1, x_2) = (x_1 x_2, x_1^2 + x_2)$, compute its Jacobian and the reverse product with $v = (2, -1)^T$. Verify the returned tensor with `autograd.grad` at a nonzero point.

Solution. The numerator-layout Jacobian has one row per output and one column per input:

$$J_f(x) = \begin{pmatrix} x_2 & x_1 \\ 2x_1 & 1 \end{pmatrix}, \quad D_x(v^T f) = v^T J_f = (2x_2 - 2x_1, 2x_1 - 1).$$

PyTorch returns the transposed entries $J_f^T v$ in the shape of the input tensor. At $x = (2, 3)^T$, this is $(2, 3)^T$.

```
import torch
dtype = torch.float64
x = torch.tensor([2., 3.], dtype=dtype, requires_grad=True)
y = torch.stack((x[0]*x[1], x[0].square()+x[1]))
v = torch.tensor([2., -1.], dtype=dtype)
(g,) = torch.autograd.grad(y, x, grad_outputs=v)
J = torch.tensor([[3., 2.], [4., 1.]], dtype=dtype)
torch.testing.assert_close(g, J.T @ v)
torch.testing.assert_close(g, torch.tensor([2., 3.], dtype=dtype))
assert x.grad is None
```

The seed v specifies the output combination whose sensitivity is required. It is held fixed during differentiation. `autograd.grad` returns the derivative directly, whereas `backward` accumulates gradients in eligible leaf tensors' `.grad` fields. Storage shape does not alter the distinction between the row derivative $D_x L$ and column gradient $\nabla_x L$.

Problem 2. The reduction determines the gradient scale

Problem. Derive the gradients of

$$L = \frac{1}{B} \|XW^T + \mathbf{1}_B b^T - T\|_F^2,$$

where $X \in \mathbb{R}^{B \times d}$, $W \in \mathbb{R}^{m \times d}$, $b \in \mathbb{R}^m$, and $T \in \mathbb{R}^{B \times m}$. Compare with averaging over all output entries.

Solution. Let $Z = XW^T + \mathbf{1}_B b^T$, $E = Z - T$, and $\Delta = 2E/B$. The differential satisfies

$$dL = \text{tr}(\Delta^T dZ), \quad dZ = dX W^T + X dW^T + \mathbf{1}_B db^T.$$

Collecting coefficients using the Frobenius inner product gives

$$\boxed{\nabla_X L = \Delta W, \quad \nabla_W L = \Delta^T X, \quad \nabla_b L = \Delta^T \mathbf{1}_B.}$$

Their shapes are (B, d) , (m, d) , and $(m, 1)$ respectively. Each row uses the same bias, so every row contributes to its derivative. That is why the bias gradient sums across observations.

The entrywise mean is $\tilde{L} = \|E\|_F^2 / (Bm) = L/m$. Every gradient is therefore divided by m as well. The dimensions of the stored parameter gradients remain unchanged.

```

gen = torch.Generator().manual_seed(23)
B, d, m = 4, 3, 2
X = torch.randn(B, d, generator=gen, dtype=dtype,
                 requires_grad=True)
W = torch.randn(m, d, generator=gen, dtype=dtype,
                 requires_grad=True)
b = torch.randn(m, generator=gen, dtype=dtype,
                 requires_grad=True)
T = torch.randn(B, m, generator=gen, dtype=dtype)
E = X @ W.T + b - T
L = E.square().sum()/B
gX, gW, gb = torch.autograd.grad(L, (X, W, b), retain_graph=True)
delta = 2*E.detach()/B
torch.testing.assert_close(gX, delta @ W.detach())
torch.testing.assert_close(gW, delta.T @ X.detach())
torch.testing.assert_close(gb, delta.sum(0))
mean_grads = torch.autograd.grad(E.square().mean(), (X, W, b))
for g, gm in zip((gX, gW, gb), mean_grads):
    torch.testing.assert_close(gm, g/m)

```

Problem 3. A detached branch

Problem. Construct a scalar computation in which one input contributes through two branches. Compare its derivative with a version detaching one branch, and with finite differences of the original numerical function.

Solution. Choose $f(x) = x^2 + 3x$, whose derivative is $2x + 3$. Detaching the linear branch preserves its numerical value but gives it derivative zero in the recorded graph. Autodiff then returns $2x$. At $x = 2$ the correct derivative is 7 and the detached result is 4.

```

x = torch.tensor(2., dtype=dtype, requires_grad=True)
(correct,) = torch.autograd.grad(x.square()+3*x, x)
(detached,) = torch.autograd.grad(x.square()+3*x.detach(), x)
f = lambda a: a*a + 3*a
h = 1e-5
finite_difference = (f(2+h)-f(2-h))/(2*h)
assert abs(float(correct)-7.) < 1e-12
assert abs(float(detached)-4.) < 1e-12
assert abs(finite_difference-7.) < 1e-8

```

The finite difference reevaluates both branches at the perturbed input. The detached graph instead treats the linear branch's evaluated value as a constant for differentiation. The discrepancy identifies the missing dependency. It is not a finite-difference approximation error: for this quadratic, the centered-difference truncation error is zero in exact arithmetic.

Problem 4. Saving parameter values

Problem. Save a module's state dictionary, load it into a newly initialized module of the same architecture, and compare outputs on fixed inputs. Explain why equal outputs do not establish exact resumption of training.

Solution. A state dictionary contains registered parameters and persistent buffers. Here a serialized in-memory buffer performs the same save/load operation as a file.

```

import io
torch.manual_seed(23)
model = torch.nn.Sequential(torch.nn.Linear(3, 4, dtype=dtype),
                             torch.nn.Tanh(),
                             torch.nn.Linear(4, 2, dtype=dtype))
model.eval()
X = torch.tensor([[1., 2., 3.], [-1., 0., 2.]], dtype=dtype)
with torch.no_grad():
    expected = model(X)
buffer = io.BytesIO()
torch.save(model.state_dict(), buffer)
buffer.seek(0)
restored = torch.nn.Sequential(torch.nn.Linear(3, 4, dtype=dtype),
                               torch.nn.Tanh(),
                               torch.nn.Linear(4, 2, dtype=dtype))
restored.load_state_dict(torch.load(buffer, weights_only=True))
restored.eval()
with torch.no_grad():
    torch.testing.assert_close(restored(X), expected, rtol=0, atol=0)

```

Equal state, architecture, and evaluation settings give equal outputs for this deterministic CPU computation. The state dictionary alone does not store the architecture’s Python definition, module training/evaluation flags, optimizer moments, learning-rate scheduler state, random-generator states, data-order position, or pending accumulated gradients. A training checkpoint must record the relevant additional state. Exact continuation also depends on the numerical environment and deterministic execution. The test above verifies restoration of this model’s evaluated function on the fixed inputs.